

COMPILER DESIGN

ASSIGNMENT

Issues In The Design of A Code
Generator

REG. NO: RA1711003010611

NAME : ANKITA PAL

SUBCODE : 15CS314J

BATCH : 1

CLASS : TP606

FAULTY : MS. G.K. SANDHIA

DATE : 18-03-2020

PAGES : 6

ISSUES IN THE DESIGN OF A CODE GENERATOR

Code generator converts the immediate representation of source code into a form that can be readily executed by the machine. A code generator is expected to generate the correct code. Designing of code generator should be done in such a way so that it can be easily implemented, tested and maintained.

The following issue arises during the code generation phase:

(1.) Input to code generator -

(i) Input to code generator is the intermediate code generated by the front end, along with information in the symbol table that determines the run-time addresses of the data-objects denoted by the names in the intermediate representation.

(ii) Intermediate codes may be represented mostly in quadruples, triples, indirect triples, postfix notation, syntax trees, DAGs, etc. The code generation phase just proceeds on an assumption that the input are free from all of syntactic and static semantic errors, the necessary type checking has taken place and

the type-conversion operators have been inserted wherever necessary.

(2.) Target program -

The target program is the output of the code generator. The output may be absolute machine language, relocatable machine language, assembly language.

- (i) Absolute machine language as output has advantages that it can be placed in a fixed memory location and can be immediately executed.
- (ii) Relocatable machine language as an output allows subprograms and subroutines to be compiled separately. Relocatable object modules can be linked together and loaded by linking loader. But there is added expense of linking and loading.
- (iii) Assembly language as output makes the code generation easier. We can generate symbolic instructions and use macro facilities of assembler in generator code. And we need an additional assembly step after code generation.

(3.) Memory Management—

Mapping the names in the source program to the addresses of data objects is done by the front end and the code generator. A name in the three address statements refers to the symbol table entry for name. Then from the table entry, a relative address can be determined for the name.

(4.) Instruction selection—

Selecting the best instructions will improve the efficiency of the program. It includes the instructions that should be complete and uniform. Instruction speeds and machine idioms also play a major role when efficiency is considered. But if we don't care about the efficiency of the target program then instruction selection is straight-forward.

For example, the respective three-address statements would be translated into the latter code sequence as shown below:

$P := Q + R$

$S := P + T$

MOV Q, R0

```
ADD R, R0
MOV R0, P
MOV P, R0
ADD T, R0
MOV R0, S
```

Hence, the fourth statement is redundant as the value of the P is loaded again in that statement that just has been stored in the previous statement. It leads to an inefficient code sequence. A given intermediate representation can be translated into many code sequences, with significant cost differences between the different implementations.

A priori knowledge of instruction cost is needed in order to design good sequences, but accurate cost information is difficult to predict.

(5.) register allocation issues—

Use of registers make the computations faster in comparison to that of memory, so efficient utilization of registers is important. The use of registers are subdivided into two subproblems:

(i) During register allocation — we select only those set of variables that will reside in the registers at each point in the program.

(ii) During a subsequent register assignment phase, the specific register is picked to access the variable.

As the number of variables increases, the optimal assignment of registers to variables becomes difficult.

Mathematically, this problem becomes NP-complete. Certain machine requires register pairs consist of an even and next odd-numbered register. For example,

M a, b

These types of multiplicative instruction involve register pairs where the multiplicand is an even register and b, the multiplier is the odd register of the even/odd register pair.

(b) Evaluation Order

The code generator decides the order in which the instruction will be executed. The order of computations affects the efficiency of the target code. Among many computational orders, some will require only few registers to hold the intermediate results. However, picking the best order in the general case is a difficult NP-complete problem.

(7.) Approaches to code generation issues :

Code generators must always generate the correct code. It is essential because of the number of special cases that a code generator might face. Some of the design goals of code generators are:

- i) correct
- ii) Easily maintainable
- iii) Testable
- iv) Efficient